

Reflexión de preguntas

Alfredo Carrera
Mario Vallatoro

1. ¿Cuál es la diferencia principal entre un sistema experto y un programa de software tradicional?

El software tradicional sigue una lista fija de instrucciones paso a paso de forma rígida. Un sistema experto imita el pensamiento de un especialista usando un conjunto de reglas ("Si pasa X, entonces haz Y") para resolver problemas complejos adaptándose a la situación.

2. ¿Por qué se dice que los sistemas expertos tienen conocimiento separado de su motor de razonamiento? ¿Cuál es la ventaja de esto?

Significa que las reglas del problema están guardadas en un lugar distinto al programa que procesa las reglas (el motor).

Ventaja: Permite actualizar, corregir o agregar nueva información sin necesidad de modificar ni reprogramar todo el sistema.

3. ¿Qué es la base de hechos y en qué se diferencia de la base de conocimiento?

Base de hechos: Contiene los datos específicos y temporales del caso particular que se está evaluando hoy (ej. *"El paciente Juan tiene 38°C de fiebre"*).

Base de conocimiento: Contiene las reglas generales y la teoría fijas del dominio (ej. *"Si hay fiebre alta y tos seca, sospechar de gripe"*).

4. ¿Qué significa que un sistema experto pueda "explicar su razonamiento"? ¿Por qué esto es importante en medicina o derecho?

Significa que el sistema puede mostrar paso a paso qué reglas utilizó y qué datos tomó en cuenta para llegar a su conclusión.

Importancia: En medicina o derecho las decisiones son críticas; los profesionales necesitan verificar la lógica del sistema para confiar en el diagnóstico o fallo antes de actuar.

5. ¿Por qué fracasaron comercialmente los sistemas expertos en los años 90? Menciona al menos 3 razones.

- **Mantenimiento costoso y complejo:** Actualizar manualmente miles de reglas resultaba difícil e ineficiente.
- **Rigidez ante imprevistos:** Si se presentaba una situación fuera de sus reglas predefinidas, el sistema fallaba por completo.

- **Falta de escalabilidad tecnológica:** La tecnología de la época era muy costosa y lenta para procesar bases de datos gigantescas.

6. Dada la siguiente regla: SI (fiebre AND tos) OR perdida_olfato ENTONCES sospecha_covid y los hechos: {fiebre=True, tos=False, perdida_olfato=True} — ¿Se activa la regla? ¿Por qué?

En la primera parte (**fiebre AND tos**) da como resultado *Falso* porque tiene fiebre pero no tos.

Sin embargo, la condición usa un **OR** con **perdida_olfato**, la cual es *Verdadera*.

Al haber al menos un camino verdadero en un **OR**, toda la condición se cumple.

7. Completa la tabla de verdad para la expresión (A AND NOT B) OR (NOT A AND B) para todos los valores posibles de A y B.

VALORES 1 = TRUE 0 = FALSE

ELEMENTO A	ELEMENTO B	NOT B	NOT A	A AND B	A AND B	(A AND B) OR (A AND B)
0	0	1	1	0	0	0
0	1	0	1	1	1	1
1	0	1	0	0	0	0
1	1	0	0	0	0	0

8. ¿Cuál es la diferencia entre encadenamiento hacia adelante y hacia atrás? Da un ejemplo de una situación real donde usarías cada uno.

Hacia adelante (Basado en datos): Empiezas con los síntomas o hechos iniciales y aplicas reglas para descubrir qué conclusión generan.

Ejemplo: Un sistema de emergencias médicas ingresa los síntomas de un paciente para determinar qué enfermedad padece.

Hacia atrás (Basado en metas): Empiezas con una hipótesis o meta final y vas buscando hacia atrás si existen las evidencias necesarias para confirmarla.

Ejemplo: Un perito técnico que investiga la causa de un incendio para confirmar si fue provocado por un cortocircuito.

9. Diseña 3 reglas IF-THEN para un sistema experto que asesore a estudiantes sobre qué lenguaje de programación aprender primero, basándose en su objetivo (desarrollo web, análisis de datos, desarrollo de videojuegos).

Regla 1: SI objetivo = "desarrollo web" **ENTONCES** recomendar = "JavaScript"

Regla 2: SI objetivo = "análisis de datos" **ENTONCES** recomendar = "Python"

Regla 3: SI objetivo = "desarrollo de videojuegos" **ENTONCES** recomendar = "C#"

10. Dibuja la red de inferencia correspondiente a las 3 reglas que diseñaste en la pregunta anterior.

[Objetivo: Desarrollo Web] -----(Regla 1)-----> [Recomendar: JavaScript]

[Objetivo: Análisis de Datos] -----(Regla 2)-----> [Recomendar: Python]

[Objetivo: Videojuegos] -----(Regla 3)-----> [Recomendar: C#]

11. ¿Qué problema de diseño podría surgir si dos reglas tienen exactamente las mismas condiciones pero conclusiones diferentes? ¿Cómo lo resolverías?

Problema: Surge un conflicto de incoherencia. El sistema entra en dilema al dar dos respuestas opuestas para una misma entrada.

Solución: Implementar una estrategia de resolución de conflictos, asignando un peso de prioridad o nivel de certeza a cada regla, o solicitando al usuario/experto que agregue una condición adicional para diferenciarlas.